

SHARIF UNIVERSITY OF TECHNOLOGY

Department of Electrical Engineering

TECHNICAL REPORT

Weekly Camera-Traffic Profiles with RDD PCA and K-Means

AUTHOR

Reza Sanaee Nezhad

Student ID: 401101502

INSTRUCTOR / INSTRUCTORS

Dr Iman Gholampour

ACADEMIC CONTEXT

Big Data Processing

Contents

1 Feature engineering using RDDs 2

2 L1 normalization and interpretation 4

3 Centering, SVD, and PCA selection 6

4 Initialization comparison and silhouette 7

5 Four visualization families and interpretation 11

6 Enriched camera statistics and exports 15

7 BFR summaries and Mahalanobis distance 18

8 Frobenius/SVD proof 18

9 Centroid-linkage agglomeration 18

10 Rank-two energy 20

11 Item-item similarity and missing rating 20

12 Cold start 20

1 Feature engineering using RDDs

The first practical exercise represents each observed camera by its weekly temporal distribution. Raw traffic strings are loaded into an RDD; DeviceCode is extracted from index 0 and PassDateTime from index 4. datetime parses YYYY-MM-DD HH:MM:SS and supplies weekday/hour. With Monday numbered 1, the feature index is (DOW-1)*24+hour. Each observation maps to ((camera,index),1); reduceByKey counts observations for each slot. The grouped slots then populate a zero-initialized vector of length 168. Missing time slots remain zero. The vector describes accumulated observations over the dataset, not necessarily one particular calendar week. The source text's reversed 861/42/32 numerals correspond to 168 dimensions, 24 hours, and an hour range of 0 to 23.

$$\lambda_i = \frac{s_i^2}{n-1}, \quad k = \min\left\{ j : \frac{\sum_{i=1}^{j} s_i^2}{\sum_i s_i^2} \geq 0.9 \right\}.$$

```python
import sys
import os
from datetime import datetime
from pyspark import SparkContext, SparkConf

os.environ['PYSPARK_PYTHON'] = sys.executable
os.environ['PYSPARK_DRIVER_PYTHON'] = sys.executable

try:
    sc.stop()
except:
    pass

conf = SparkConf().setAppName("TrafficFeatureEngineering").setMaster("local[*]")
sc = SparkContext(conf=conf)

traffic_path = "traffic.csv"
raw_rdd = sc.textFile(traffic_path)

header = raw_rdd.first()
data_rdd = raw_rdd.filter(lambda line: line != header)

def parse_and_extract_features(line):
    try:
        parts = line.split(',')
        device_code = parts[0].strip()
        time_str = parts[4].strip()

        dt_obj = datetime.strptime(time_str, "%Y-%m-%d %H:%M:%S")
```

Figure 1: Implementation and output: Feature engineering using RDDs.

```
        hour = dt_obj.hour
        dow = dt_obj.isoweekday()

        vector_idx = (dow - 1) * 24 + hour

        return ((device_code, vector_idx), 1)
    except:
        return None

features_rdd = data_rdd.map(parse_and_extract_features).filter(lambda x: x is not None)

slots_counts_rdd = features_rdd.reduceByKey(lambda a, b: a + b)

mapped_slots = slots_counts_rdd.map(lambda x: (x[0][0], (x[0][1], x[1])))

grouped_cameras = mapped_slots.groupByKey()

def create_168_vector(data_iterable):
    vector = [0] * 168
    for idx, count in data_iterable:
        if 0 <= idx < 168:
            vector[idx] = count
    return vector

final_vectors_rdd = grouped_cameras.mapValues(create_168_vector)
final_vectors_rdd.persist()

print(f"Total Cameras Processed: {final_vectors_rdd.count()}")
for device, vec in final_vectors_rdd.take(2):
    print(f"Device: {device}")
    non_zero_slots = [(i, v) for i, v in enumerate(vec) if v > 0]
```

Figure 2: Implementation and output: Feature engineering using RDDs.

```
Total Cameras Processed: 947
Device: 100700853
Non-zero slots: [(0, 1387), (1, 788), (2, 383), (3, 241), (4, 274)] ...
Device: 113
Non-zero slots: [(0, 25), (1, 10), (2, 2), (3, 3), (4, 1)] ...
```

Figure 3: Diagram or recorded result: Feature engineering using RDDs.

```
def l1_normalize(vector):
    total_sum = sum(vector)
    if total_sum == 0:
        return vector
    return [x / total_sum for x in vector]

normalized_rdd = final_vectors_rdd.mapValues(l1_normalize)
normalized_rdd.persist()

print(f"Total Normalized Vectors: {normalized_rdd.count()}")

sample = normalized_rdd.take(1)[0]
device_id = sample[0]
vec = sample[1]

print(f"Device: {device_id}")
print(f"Sum of vector elements: {sum(vec)}")
print(f"Vector preview: {vec[:10]}...")
```

```
Total Normalized Vectors: 947
Device: 100700853
Sum of vector elements: 0.9999999999999998
Vector preview: [0.004555861031457448, 0.0025883334482973822, 0.001258035165860276, 0.0007916095952280065, 0.0009000042700932523, 0.00196424
28960429375, 0.006500395804797614, 0.010540560958865863, 0.011006986529498133, 0.012084363903916333]...
```

Figure 4: Implementation and output: Feature engineering using RDDs.

2 L1 normalization and interpretation

mapValues divides each nonzero vector by its total. The components then sum to one. This compares temporal shape rather than absolute traffic volume: a camera observing 100 passages and another observing 10000 passages can have similar normalized profiles if their relative hourly distribution is similar. Keep the raw volume as a separate descriptive feature. A zero-total vector needs an explicit guard rather than division by zero. Low-count cameras also have noisier normalized profiles; equal normalization does not remove uncertainty from sparse sampling.

```
k_pca = np.searchsorted(cumulative_variance, 0.90) + 1

print(f"Selected k_pca for 90% variance: {k_pca}")
print(f"Cumulative Variance at k={k_pca}: {cumulative_variance[k_pca-1]:.4f}")

plt.figure(figsize=(10, 6))
plt.plot(range(1, 169), cumulative_variance, marker='.', linestyle='--')
plt.axhline(y=0.90, color='r', linestyle='-', label='90% Threshold')
plt.axvline(x=k_pca, color='g', linestyle='-', label=f'k_pca={k_pca}')
plt.xlabel('Number of Components')
plt.ylabel('Cumulative Explained Variance')
plt.title('PCA Analysis: Cumulative Explained Variance')
plt.legend()
plt.grid(True)
plt.savefig('pca_variance_plot.png')
print("Plot saved as pca_variance_plot.png")

V = svd.V.toArray()
V_k = V[:, :k_pca]
bc_Vk = sc.broadcast(V_k)

def transform_to_pca(v):
    return np.dot(v, bc_Vk.value)

pca_transformed_rdd = centered_rdd.map(transform_to_pca)

print(f"\nFinal Transformed RDD Shape: {pca_transformed_rdd.count()} rows x {k_pca} columns")
sample_pca = pca_transformed_rdd.take(1)[0]
print(f"Sample PCA Vector (First 5 components): {sample_pca[:5]}")
```

Figure 5: Implementation and output: L1 normalization and interpretation.

```
import numpy as np
import matplotlib.pyplot as plt
from pyspark.mllib.linalg import Vectors
from pyspark.mllib.linalg.distributed import RowMatrix
from pyspark.mllib.stat import Statistics

vector_rdd = normalized_rdd.map(lambda x: Vectors.dense(x[1]))
vector_rdd.cache()

summary = Statistics.colStats(vector_rdd)
mean_vector = summary.mean()
bc_mean = sc.broadcast(mean_vector)

def center_vector(v):
    return Vectors.dense(v - bc_mean.value)

centered_rdd = vector_rdd.map(center_vector)
centered_rdd.cache()

mat = RowMatrix(centered_rdd)
svd = mat.computeSVD(168, computeU=False)

s = svd.s.toArray()
eigenvalues = s ** 2
total_variance = np.sum(eigenvalues)
explained_variance_ratio = eigenvalues / total_variance
cumulative_variance = np.cumsum(explained_variance_ratio)
```

Figure 6: Implementation and output: L1 normalization and interpretation.

3 Centering, SVD, and PCA selection

Statistics.colStats computes the per-feature mean. Broadcasting the mean permits every camera vector to be centered consistently. SVD of the centered matrix yields singular values and right singular vectors. PCA covariance eigenvalues are s_i^2/(n-1), for n cameras and sample covariance; the denominator cancels in explained-variance ratios. Singular-value squares are proportional to the sample-covariance eigenvalues, with a common factor of 1/(n-1). Cumulative explained variance is calculated from squared singular values, and the first component count reaching 0.90 is selected. The right singular vectors supply the projection matrix; centered vectors are multiplied by the first k_pca columns. The submission prints the selected dimension and displays the variance plot. The narrative does not contain a reliably extractable numeric k_pca, so this edition retains the exact output image instead of inserting a guessed value. PCA preserves a stated fraction of variance, not every kind of task-relevant information.

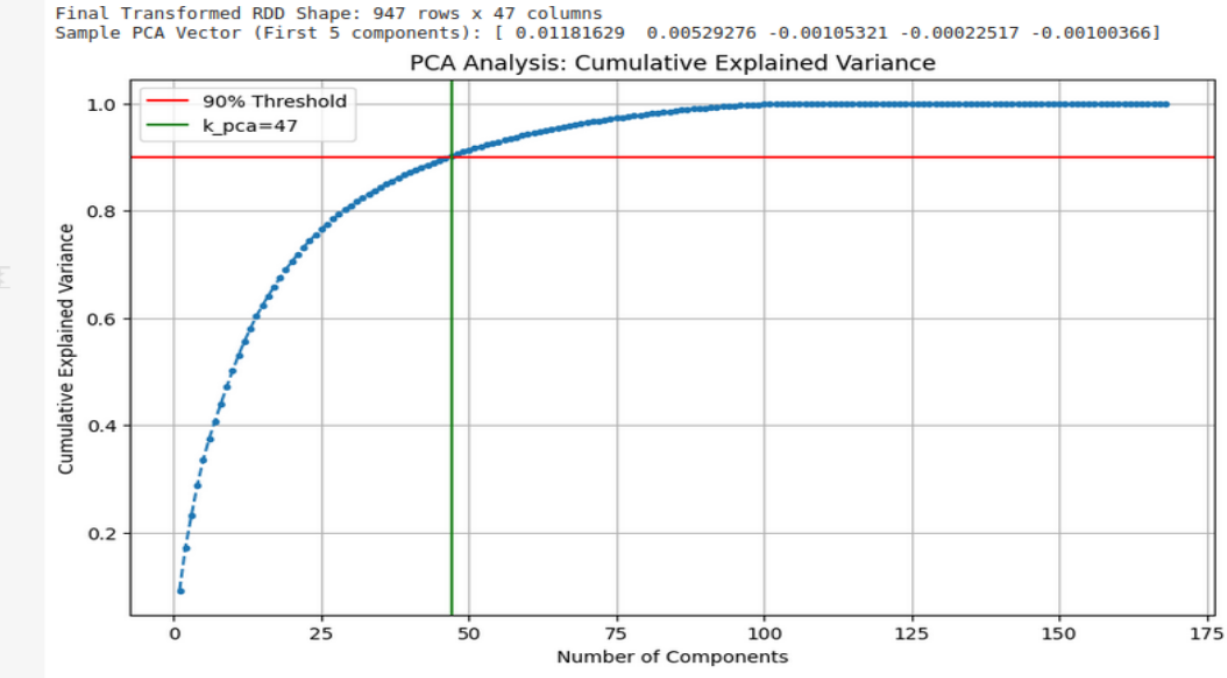


Figure 7: Diagram or recorded result: Centering, SVD, and PCA selection.

4 Initialization comparison and silhouette

K is varied from 3 to 12 for random and the requested spread-based initialization. The silhouette for point i is s(i)=(b(i)-a(i))/max(a(i),b(i)), where a is average within-cluster distance and b is the minimum mean distance to another cluster. Scores near one indicate good cohesion/separation under that distance. Singleton conventions and distance choices matter. The report describes evaluating a random sample capped at 10000 to avoid excessive memory use. Sampling can make evaluation practical, but the central limit theorem does not guarantee that rare clusters are represented or that the chosen K is correct. Report sample size, seed, and cluster coverage; repeated samples are useful for sensitivity analysis. Spread-based initialization can improve the chance of a favorable start, but cannot guarantee faster convergence, repeatability without a fixed seed, or a global optimum. Spark's k-means|| is a scalable initializer and should be distinguished from exact sequential K-Means++ when that mode is used. The source contains the full score/output tables for both approaches.

```
from pyspark.mllib.clustering import KMeans
from sklearn.metrics import silhouette_score
import numpy as np
import matplotlib.pyplot as plt

# Sample 10,000 records for silhouette evaluation; full-data evaluation can be expensive.
sample_count = 10000
total_count = pca_transformed_rdd.count()

if total_count > sample_count:
    fraction = sample_count / total_count
    # Take a random sample
    sampled_rdd = pca_transformed_rdd.sample(False, fraction, 42)
    X_local = np.array(sampled_rdd.collect())
else:
    X_local = np.array(pca_transformed_rdd.collect())

print(f"Data sampled for evaluation: {len(X_local)} records")

k_values = range(3, 13)
results = []
init_modes = [("KMeans (Random)", "random"), ("KMeans++", "k-means||")]

print(f"{'Method':<20} | {'K':<3} | {'Silhouette Score':<20}")
print("-" * 50)

best_k = 0
best_score = -1
best_method = ""
```

Figure 8: Implementation and output: Initialization comparison and silhouette.

```
for method_name, mode in init_modes:
    scores = []
    for k in k_values:

        model = KMeans.train(pca_transformed_rdd, k, maxIterations=100, initializationMode=mode, seed=42)
        temp_rdd = sc.parallelize(X_local)
        predictions = model.predict(temp_rdd)
        labels = predictions.collect()


        if len(set(labels)) > 1:
            score = silhouette_score(X_local, labels)
        else:
            score = -1

        scores.append(score)

        print(f"{method_name:<20} | {k:<3} | {score:.4f}")

        if score > best_score:
            best_score = score
            best_k = k
            best_method = method_name

    results.append((method_name, scores))

print("-" * 50)
print(f"Best Result: K={best_k} with Score={best_score:.4f} using {best_method}")
```

Figure 9: Implementation and output: Initialization comparison and silhouette.

```
plt.figure(figsize=(10, 6))
for method_name, scores in results:
    plt.plot(k_values, scores, marker='o', label=method_name)

plt.xlabel('Number of Clusters (K)')
plt.ylabel('Silhouette Score (Sampled)')
plt.title('KMeans vs KMeans++ Silhouette Analysis')
plt.legend()
plt.grid(True)
plt.savefig('kmeans_silhouette_comparison.png')
```

Figure 10: Diagram or recorded result: Initialization comparison and silhouette.

Data sampled for evaluation: 947 records

Method	K	Silhouette Score
KMeans (Random)	3	0.4409
KMeans (Random)	4	0.3161
KMeans (Random)	5	0.3370
KMeans (Random)	6	0.0751
KMeans (Random)	7	0.0715
KMeans (Random)	8	0.0681
KMeans (Random)	9	0.1039
KMeans (Random)	10	0.0560
KMeans (Random)	11	0.0739
KMeans (Random)	12	0.0292
KMeans++	3	0.4728
KMeans++	4	0.2833
KMeans++	5	0.3459

Figure 11: Diagram or recorded result: Initialization comparison and silhouette.

KMeans++	6	0.3227
KMeans++	7	0.0781
KMeans++	8	0.2748
KMeans++	9	0.2689
KMeans++	10	0.0741
KMeans++	11	0.1999
KMeans++	12	0.1071

Figure 12: Diagram or recorded result: Initialization comparison and silhouette.

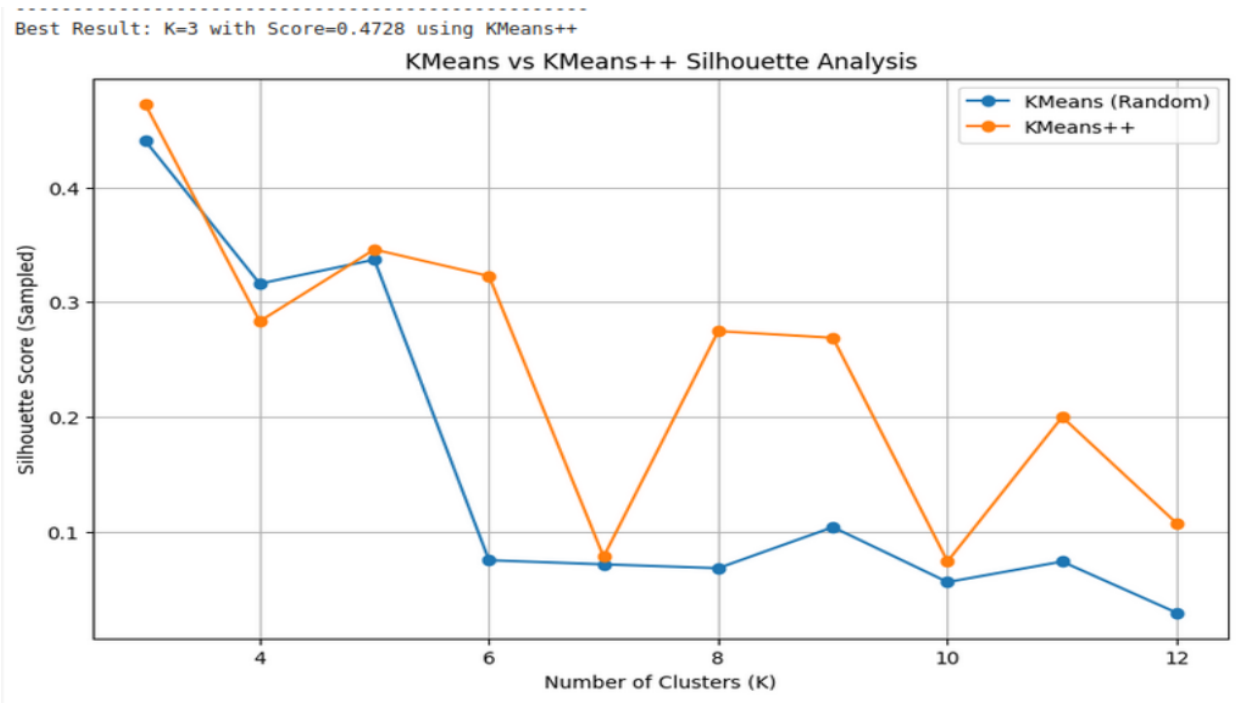


Figure 13: Diagram or recorded result: Initialization comparison and silhouette.

5 Four visualization families and interpretation

Cluster-size bar charts reveal how many cameras receive each label. Unequal sizes may reflect actual class imbalance; they are not by themselves proof of a poor clustering. Mean 24-hour line profiles are obtained by aggregating the 168 slots across weekdays, exposing possible morning/evening peaks or flatter profiles. A PC1/PC2 scatter plot is a visual projection of the fitted data; apparent overlap in two dimensions does not automatically invalidate separation in the full fitted PCA space. For each cluster, the closest camera to its centroid is selected as a representative. Comparing that camera's hourly profile with the mean makes the average more interpretable. A good match is useful descriptive evidence, but does not prove that every member has the same behavior or that the mean is a unique physical traffic archetype. The experiment report favors the spread-based initialization based on its plots and scores; that conclusion remains restricted to this submitted run.

```
import matplotlib.pyplot as plt
import numpy as np

def get_24h_profile(vector_168):
    matrix = np.array(vector_168).reshape(7, 24)
    return np.mean(matrix, axis=0)

combined_rdd = pca_transformed_rdd.zip(final_vectors_rdd.map(lambda x: x[1]))
labeled_rdd = combined_rdd.map(lambda x: (model.predict(x[0]), (x[0], x[1])))
labeled_rdd.persist()

cluster_counts = labeled_rdd.countByKey()
sorted_clusters = sorted(cluster_counts.items())

cluster_centers = model.clusterCenters

stats = []
for k in range(best_k):
    cluster_data = labeled_rdd.filter(lambda x: x[0] == k)

    avg_vec = cluster_data.map(lambda x: np.array(x[1][1])).reduce(lambda a, b: a + b)
    avg_vec = avg_vec / cluster_counts[k]

    center_vec = cluster_centers[k]

    def get_dist(record):
        pca_vec = record[1][0]
        dist = np.linalg.norm(pca_vec - center_vec)
        return (dist, record[1][1])

    rep_camera = cluster_data.map(get_dist).min(key=lambda x: x[0])
```

Figure 14: Implementation and output: Four visualization families and interpretation.

```
    stats.append({
        'cluster_id': k,
        'count': cluster_counts[k],
        'avg_profile': get_24h_profile(avg_vec),
        'rep_profile': get_24h_profile(rep_camera[1])
    })

plt.figure(figsize=(10, 6))
clusters = [s['cluster_id'] for s in stats]
counts = [s['count'] for s in stats]
plt.bar(clusters, counts, color='skyblue', edgecolor='black')
plt.title('1. Cluster Size Distribution')
plt.xlabel('Cluster ID')
plt.ylabel('Number of Cameras')
plt.xticks(clusters)
plt.savefig('plot_1_bar.png')
plt.show()

plt.figure(figsize=(12, 6))
for s in stats:
    plt.plot(s['avg_profile'], label=f"Cluster {s['cluster_id']}")
plt.title('2. Average 24-Hour Profile per Cluster')
plt.xlabel('Hour of Day (0-23)')
plt.ylabel('Normalized Traffic')
plt.legend()
plt.grid(True)
plt.savefig('plot_2_profiles.png')
plt.show()
```

Figure 15: Diagram or recorded result: Four visualization families and interpretation.

```
for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.savefig('plot_4_representatives.png')
plt.show()
```

Figure 16: Diagram or recorded result: Four visualization families and interpretation.

```
sample_data = labeled_rdd.sample(False, 0.1, 42).collect()
pca_x = [x[1][0][0] for x in sample_data]
pca_y = [x[1][0][1] for x in sample_data]
labels = [x[0] for x in sample_data]

plt.figure(figsize=(10, 8))
scatter = plt.scatter(pca_x, pca_y, c=labels, cmap='tab10', alpha=0.6, s=10)
plt.title('3. PCA Scatter Plot (PC1 vs PC2)')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.colorbar(scatter, label='Cluster ID')
plt.grid(True)
plt.savefig('plot_3_scatter.png')
plt.show()

num_clusters = len(stats)
cols = 2
rows = (num_clusters + 1) // 2
fig, axes = plt.subplots(rows, cols, figsize=(15, 4 * rows))
axes = axes.flatten()

for i, s in enumerate(stats):
    ax = axes[i]
    ax.plot(s['avg_profile'], 'b--', label='Cluster Avg', linewidth=2)
    ax.plot(s['rep_profile'], 'r-', label='Representative (Centroid)', linewidth=1.5)
    ax.set_title(f"Cluster {s['cluster_id']} (Size: {s['count']})")
    ax.set_xlabel("Hour")
    ax.legend()
    ax.grid(True)
```

Figure 17: Diagram or recorded result: Four visualization families and interpretation.

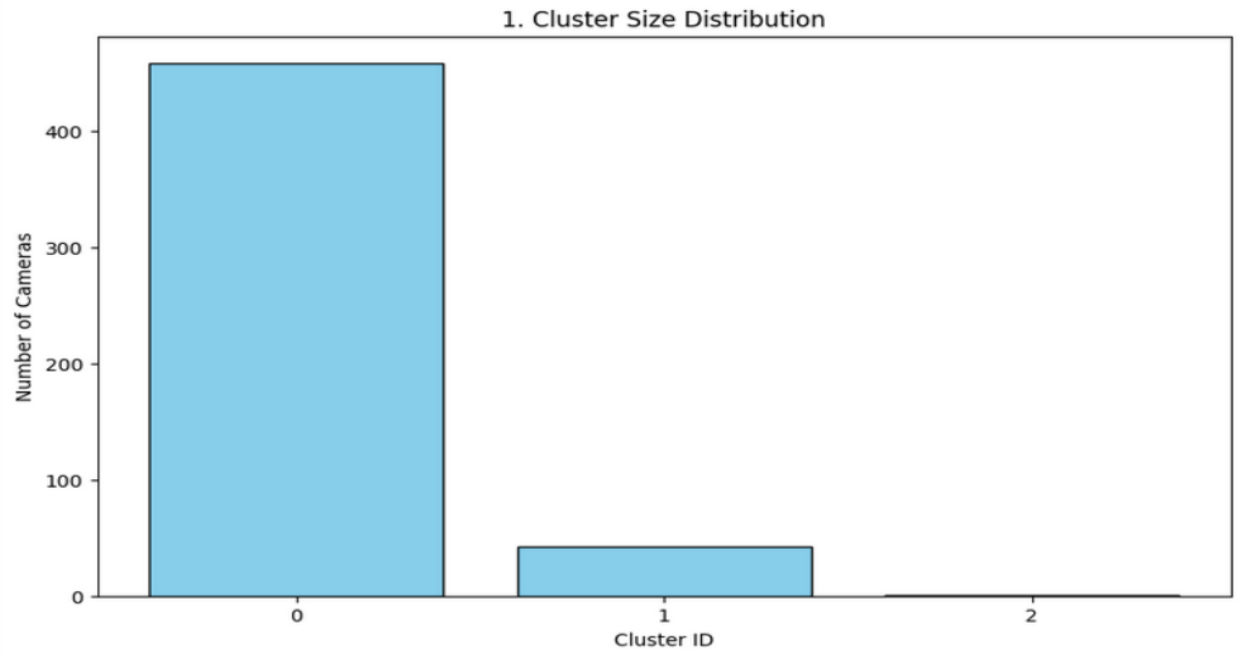


Figure 18: Diagram or recorded result: Four visualization families and interpretation.

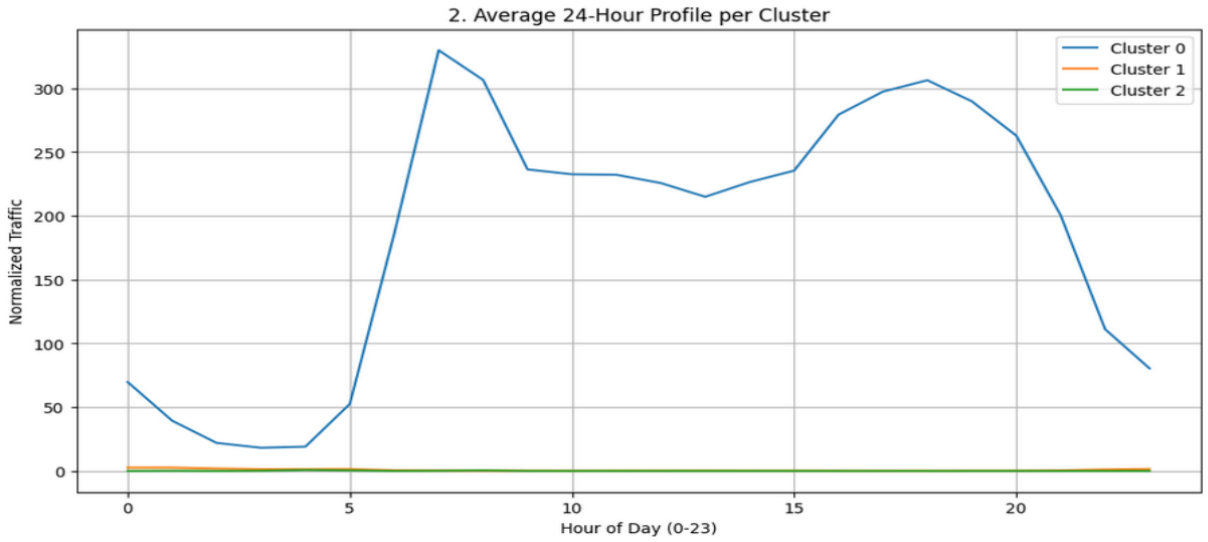


Figure 19: Diagram or recorded result: Four visualization families and interpretation.

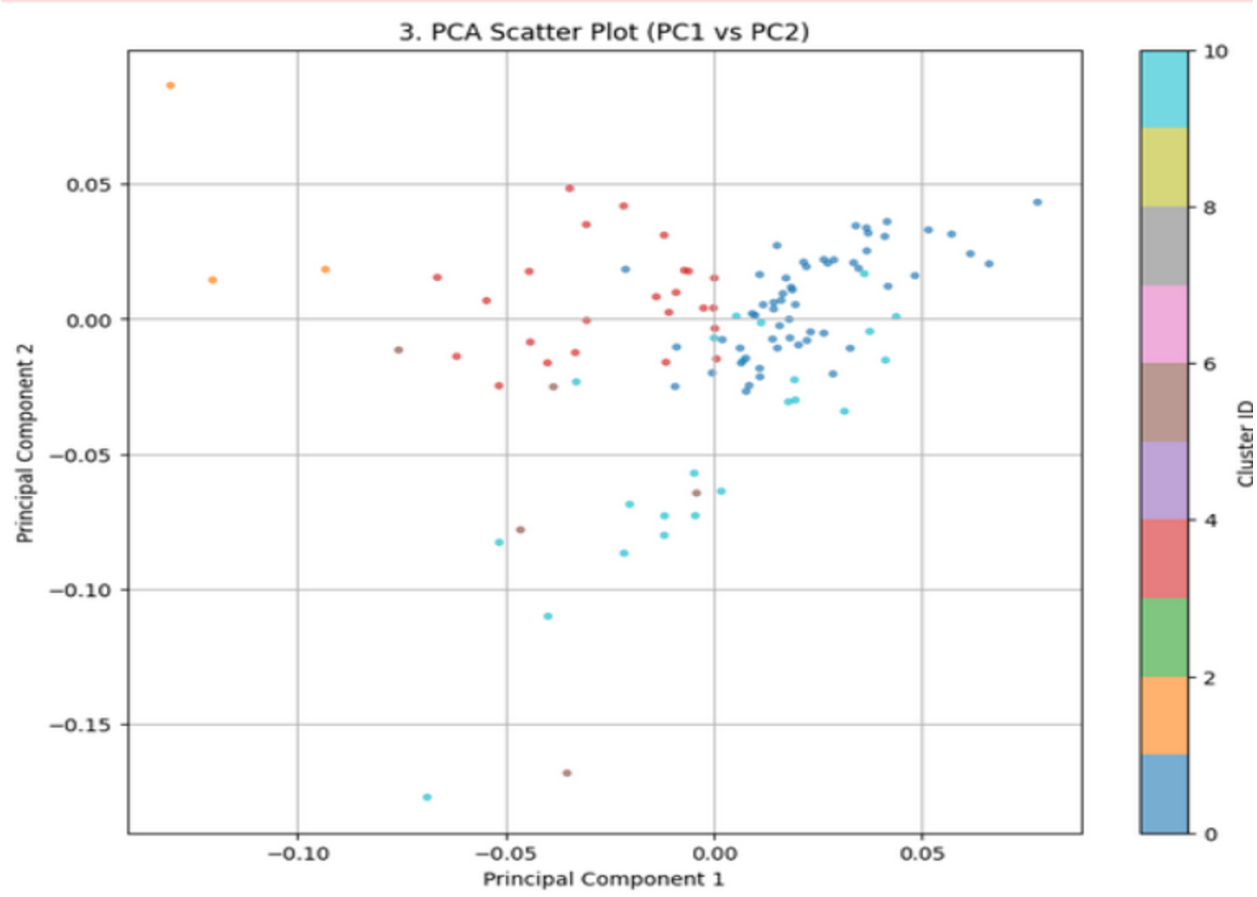


Figure 20: Diagram or recorded result: Four visualization families and interpretation.

6 Enriched camera statistics and exports

Raw records supply TotalPasses through counting by camera, UniquePlates through distinct camera/plate pairs, and PeakHour through hourly count comparisons. Cluster labels are joined to these fields. Each camera's 24-hour distribution is normalized and the cluster profile averages those distributions, avoiding unintended weighting by high-volume cameras. The report exports an Analysis_Results.xlsx workbook with four camera/profile sheets corresponding to the two initializations. Available CSV inputs are in the code directory under their required relative paths. The workbook/screenshots are submitted outputs, not newly regenerated results. The pipeline demonstrates low-level Spark feature construction, normalization, PCA, clustering, and interpretable export; it does not establish a causal classification of neighborhoods or independently validated road semantics.

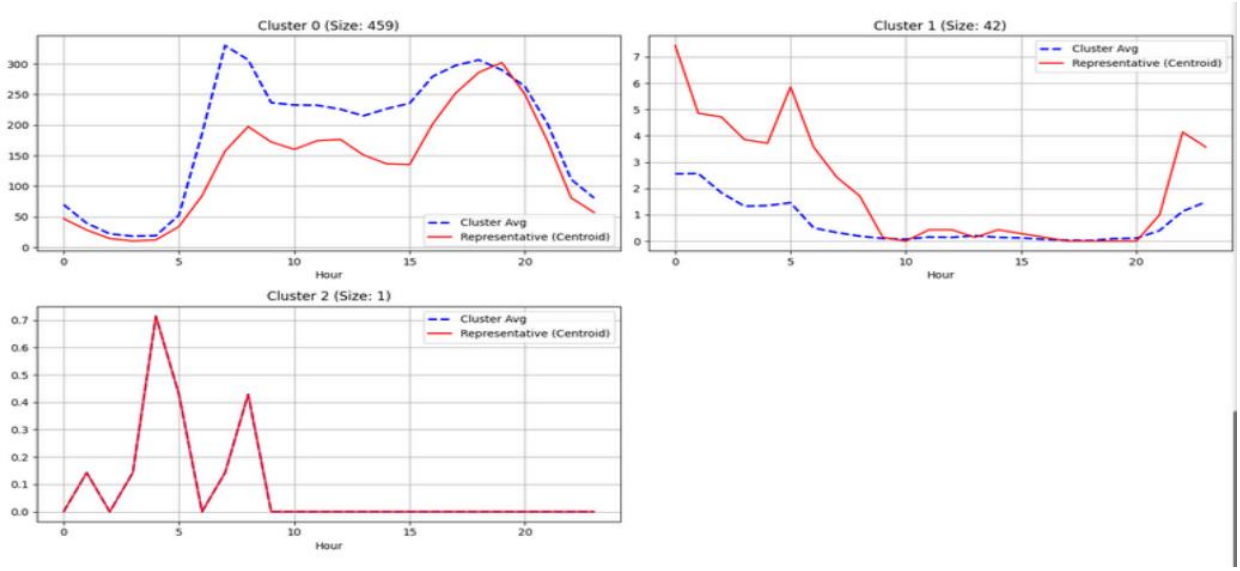


Figure 21: Diagram or recorded result: Enriched camera statistics and exports.

```
import pandas as pd
import numpy as np
from operator import add
from pyspark.mllib.clustering import KMeans


def parse_for_stats(line):
    try:
        parts = line.split(',')
          # Column mapping: 0=Device, 4=Time, 6=Plate
        return (parts[0].strip(), parts[4].strip(), parts[6].strip())
    except:
        return None

parsed_rdd = data_rdd.map(parse_for_stats).filter(lambda x: x is not None)
parsed_rdd.cache()
print(f"parsed_rdd created with {parsed_rdd.count()} records.")


total_passes = parsed_rdd.map(lambda x: (x[0], 1)).reduceByKey(add)

unique_plates = parsed_rdd.map(lambda x: (x[0], x[2])) \
                          .distinct() \
                          .map(lambda x: (x[0], 1)) \
                          .reduceByKey(add)

def get_hour_key(record):
    try:
        h = int(record[1].split(' ')[1].split(':')[0])
        return ((record[0], h), 1)
    except:
        return None
```

Figure 22: Implementation and output: Enriched camera statistics and exports.

```
hourly_counts = parsed_rdd.map(get_hour_key).filter(lambda x: x is not None).reduceByKey(add)

peak_hours = hourly_counts.map(lambda x: (x[0][0], (x[0][1], x[1]))) \
                          .reduceByKey(lambda a, b: a if a[1] > b[1] else b) \
                          .mapValues(lambda x: x[0])

stats_joined = total_passes.join(unique_plates).join(peak_hours)
camera_stats_rdd = stats_joined.mapValues(lambda x: (x[0][0], x[0][1], x[1]))

def generate_outputs(model_instance, k):
    devices = final_vectors_rdd.map(lambda x: x[0])
    device_pca = devices.zip(pca_transformed_rdd)

    cluster_preds = device_pca.map(lambda x: (x[0], model_instance.predict(x[1])))

    cam_cluster_data = cluster_preds.join(camera_stats_rdd) \
                                    .map(lambda x: (x[0], x[1][0], x[1][1][0], x[1][1][1], x[1][1][2])) \
                                    .collect()

    df_clusters = pd.DataFrame(cam_cluster_data, columns=['DeviceCode', 'ClusterID', 'TotalPasses', 'UniquePlates', 'PeakHour'])

    def to_24h_norm(vec_168):
        arr = np.array(vec_168).reshape(7, 24)
        sum_24 = np.sum(arr, axis=0)
        total = np.sum(sum_24)
        if total > 0:
            return sum_24 / total
        return sum_24

    dev_vec_cluster = final_vectors_rdd.join(cluster_preds)
```

Figure 23: Implementation and output: Enriched camera statistics and exports.

```
    cluster_profiles = dev_vec_cluster.map(lambda x: (x[1][1], (to_24h_norm(x[1][0]), 1))) \
                                      .reduceByKey(lambda a, b: (a[0] + b[0], a[1] + b[1])) \
                                      .mapValues(lambda x: x[0] / x[1]) \
                                      .collect()

    profile_rows = []
    for cid, profile in cluster_profiles:
        for h in range(24):
            profile_rows.append({'ClusterID': cid, 'Hour': h, 'AvgPHour': profile[h]})

    df_profiles = pd.DataFrame(profile_rows)

    return df_clusters, df_profiles

try:
    k_target = best_k
except NameError:
    k_target = 5


model_km = KMeans.train(pca_transformed_rdd, k_target, maxIterations=100, initializationMode="random", seed=42)
df_km_cam, df_km_prof = generate_outputs(model_km, k_target)

model_kmpp = KMeans.train(pca_transformed_rdd, k_target, maxIterations=100, initializationMode="k-means||", seed=42)
df_kmpp_cam, df_kmpp_prof = generate_outputs(model_kmpp, k_target)

df_km_cam.to_csv("kmeans_camera_clusters.csv", index=False)
df_km_prof.to_csv("kmeans_cluster_hour_profile.csv", index=False)
```

Figure 24: Diagram or recorded result: Enriched camera statistics and exports.

```
with pd.ExcelWriter('Analysis_Results.xlsx') as writer:
    df_km_cam.to_excel(writer, sheet_name='KM_CameraClusters', index=False)
    df_km_prof.to_excel(writer, sheet_name='KM_HourProfile', index=False)
    df_kmpp_cam.to_excel(writer, sheet_name='KMpp_CameraClusters', index=False)
    df_kmpp_prof.to_excel(writer, sheet_name='KMpp_HourProfile', index=False)

print("Files generated successfully.")
```

Figure 25: Implementation and output: Enriched camera statistics and exports.

7 BFR summaries and Mahalanobis distance

The discard set contains points assigned confidently to a main cluster; keep N, SUM, and SUMSQ rather than each point. The compression set summarizes groups close to one another but not yet confidently assigned to a main cluster. The retained set stores unresolved isolated points explicitly. DS/CS reduce RAM, while RS may grow if the clustering assumptions poorly fit the data. BFR's diagonal model uses axis-aligned Gaussian clusters, not arbitrary rotated covariance ellipses. The distance to centroid c is the square root of the sum over dimensions of ((x_i-c_i)/sigma_i)^2. Scaling by standard deviations accounts for unequal spread along different axes. Zero or very small variance requires a defined numerical policy. Euclidean K-Means tends to favor comparable spherical distances; Mahalanobis scaling makes an axis with a large natural spread contribute less for the same absolute offset.

8 Frobenius/SVD proof

For a reduced SVD A=U Sigma V-transpose with orthonormal columns, ||A||_F^2=trace(A-transpose A). Substitute the decomposition to obtain trace(V Sigma-transpose U-transpose U Sigma V-transpose). Since U-transpose U=I, cyclic trace rearrangement gives trace(Sigma-transpose Sigma V-transpose V)=trace(Sigma-transpose Sigma)=sum_i sigma_i^2. This argument works for rectangular matrices; Sigma need not be treated as a square matrix unless using the compact square diagonal factor.

9 Centroid-linkage agglomeration

Start with six singleton points 2,5,9,15,16,18. Merge D/E at distance 1, producing centroid 15.5. Merge DE/F at distance 2.5, producing centroid 49/3. Merge A/B at distance 3, producing centroid 3.5. Merge AB/C at distance 5.5, producing centroid 16/3. The two final clusters are {A,B,C} and {D,E,F}. The merge table defines the requested dendrogram:

Step	Merge	Height	Resulting centroid
1	D with E	1	15.5
2	DE with F	2.5	16.333333
3	A with B	3	3.5
4	AB with C	5.5	5.333333

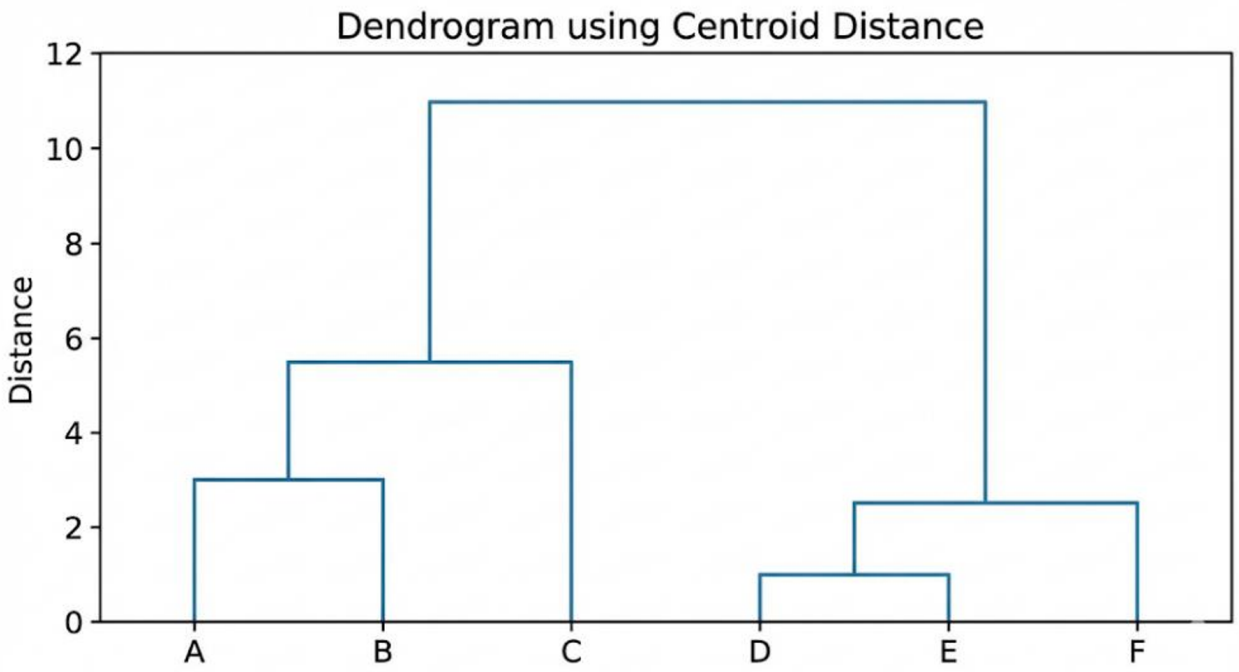


Figure 26: Diagram or recorded result: Centroid-linkage agglomeration.

10 Rank-two energy

For singular values 20,12,8,1.5,0.5, total squared energy is 400+144+64+2.25+0.25=610.5. The leading two retain 544, or 544/610.5*100=89.1072891 percent. The residual squared Frobenius error of the truncated rank-two SVD is 66.5. Energy fraction describes squared matrix norm; it should not be mistaken for an arbitrary task accuracy percentage.

11 Item-item similarity and missing rating

User means over observed scores are A=10/3, B=14/3, C=3. Centered rows are A:(2/3,unknown,5/3,-7/3), B:(1/3,1/3,-2/3,unknown), C:(unknown,1,unknown,-1). Under the assignment's user-centered cosine convention on co-rated users, item 2 has similarity +1 with item 1, -1 with item 3, and -1 with item 4. Each pair has only one common user. Ordinary Pearson correlation on a single paired observation is undefined; the calculation here is adjusted cosine under the prescribed centering, not a statistically reliable Pearson estimate. The weighted deviation estimate is mean_A + [1*(2/3)-1*(5/3)-1*(-7/3)]/(1+1+1)=10/3+4/9=34/9=3.7777778. The prediction is approximately 3.77; using exact fractions avoids accumulated rounding. Such single-overlap similarities would need significance weighting or a minimum co-rating count in a real system.

12 Cold start

An unseen user's preferences and an unseen item's appeal cannot be learned from an empty interaction history alone. Similarity denominators may be zero or latent factors absent. Popularity recommendations provide a general fallback. Content metadata, onboarding preferences, and hybrid models provide alternative information. Their effectiveness requires evaluation; they do not guarantee personalized accuracy for every new entity.